

Comprehensive Guide to Double Pendulum Simulation & Deterministic Chaos on Mobile

Pocket Lab — Episode 01

Author: **Poria Azadi** | Telegram: [@the_maze2022](#)

1 Scientific Background & Physical Intuition

In classical Newtonian mechanics, the physical universe is fundamentally deterministic: given the precise initial positions and velocities of all constituent particles, their trajectories can, in principle, be computed indefinitely into the future. This mechanistic worldview dominated scientific philosophy for centuries until the formal discovery of **Deterministic Chaos** fundamentally transformed our understanding of dynamical systems.

Chaotic systems are neither stochastic nor random. Quantum probabilities and chance play no role in their governing dynamics; their motion is dictated by exact, deterministic mathematical equations. Nevertheless, their long-term evolution remains practically unpredictable.

What Is a Double Pendulum and Why Is It Chaotic?

A simple planar pendulum exhibits regular, predictable harmonic oscillation with a well-defined period. However, attaching a second pendulum bob to the end of the first via a frictionless joint creates a **Double Pendulum**. In this configuration:

- The motion of the primary arm continuously exerts kinetic forces, angular accelerations, and torques on the secondary arm, and vice versa.
- This mutual dynamical coupling renders the governing differential equations profoundly **non-linear**.
- The system possesses **no closed-form analytical solution**. There is no general mathematical equation into which time t can be substituted to directly yield the angular positions of the bobs. The only way to reveal the future of the system is through step-by-step numerical integration.

The Butterfly Effect and Sensitive Dependence on Initial Conditions

The definitive signature of chaotic systems is **exponential divergence in phase space**. If two identical double pendulums are released simultaneously with an infinitesimal difference in their initial angles—such as 1.0° (or even 0.001 rad)—their behavior unfolds through distinct phases:

1. During the initial oscillations, both systems trace nearly identical trajectories.
2. Over time, the angular separation $\Delta\theta(t)$ grows exponentially, characterized by the system's **Lyapunov Exponent** (λ):

$$\Delta\theta(t) \sim \Delta\theta_0 e^{\lambda t} \quad (1)$$

3. Beyond a critical temporal threshold, cross-correlation drops to zero, and the two systems diverge into completely unrelated paths. This fundamental property explains why atmospheric models cannot produce accurate deterministic weather forecasts beyond approximately two weeks.

2 Mathematical Formulation & Numerical Methods

Derivation via Lagrangian Mechanics

To describe the motion of a double pendulum with arm lengths L_1, L_2 and point masses M_1, M_2 , we employ Lagrangian mechanics. Choosing the deflection angles θ_1 and θ_2 relative to the downward vertical as generalized coordinates:

The Cartesian coordinates of the primary mass:

$$x_1 = L_1 \sin \theta_1, \quad y_1 = -L_1 \cos \theta_1 \quad (2)$$

The Cartesian coordinates of the secondary mass:

$$x_2 = x_1 + L_2 \sin \theta_2, \quad y_2 = y_1 - L_2 \cos \theta_2 \quad (3)$$

The Lagrangian ($\mathcal{L}$) of the system represents the difference between total kinetic energy (T) and potential energy (V):

$$\mathcal{L} = T - V \quad (4)$$

Applying the Euler-Lagrange equations for both generalized coordinates:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\theta}_i} \right) - \frac{\partial \mathcal{L}}{\partial \theta_i} = 0 \quad (i = 1, 2) \quad (5)$$

Yields the coupled second-order differential equations for angular accelerations ($\ddot{\theta}_1 = \alpha_1$ and $\ddot{\theta}_2 = \alpha_2$) expressed in terms of angular positions and velocities ($\dot{\theta}_1 = \omega_1, \dot{\theta}_2 = \omega_2$):

$$\alpha_1 = \frac{-g(2M_1 + M_2) \sin \theta_1 - M_2 g \sin(\theta_1 - 2\theta_2) - 2 \sin(\theta_1 - \theta_2) M_2 (\omega_2^2 L_2 + \omega_1^2 L_1 \cos(\theta_1 - \theta_2))}{L_1 (2M_1 + M_2 - M_2 \cos(2\theta_1 - 2\theta_2))} \quad (6)$$

$$\alpha_2 = \frac{2 \sin(\theta_1 - \theta_2) (\omega_1^2 L_1 (M_1 + M_2) + g(M_1 + M_2) \cos \theta_1 + \omega_2^2 L_2 M_2 \cos(\theta_1 - \theta_2))}{L_2 (2M_1 + M_2 - M_2 \cos(2\theta_1 - 2\theta_2))} \quad (7)$$

Numerical Integration via 4th-Order Runge-Kutta (RK4)

Because no closed-form solution exists, we transform the system into four first-order differential equations:

$$\mathbf{y} = \begin{bmatrix} \theta_1 \\ \omega_1 \\ \theta_2 \\ \omega_2 \end{bmatrix}, \quad \frac{d\mathbf{y}}{dt} = \begin{bmatrix} \omega_1 \\ \alpha_1(\mathbf{y}) \\ \omega_2 \\ \alpha_2(\mathbf{y}) \end{bmatrix} \quad (8)$$

To mitigate numerical drift and enforce conservation of mechanical energy over extended simulations, we employ the classical **4th-order Runge-Kutta (RK4)** integration scheme. Furthermore, a **sub-stepping** architecture is implemented: each animation frame is divided into 5 discrete micro-integration intervals to maintain strict numerical stability:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4) \quad (9)$$

3 Environment Setup & Prerequisites

Running this simulation requires no specialized desktop hardware or supercomputing clusters; standard smartphone processors operating under open-source runtime environments provide sufficient computational performance.

Native Path: Termux on Android

Enter the native Ubuntu userland environment previously configured via PRoot:

```
proot-distro login ubuntu
```

Install the requisite computational libraries, graphics rendering packages, and image processing modules:

```
apt update && apt install python3 python3-numpy python3-matplotlib
python3-pil -y
```

Cloud Path Alternative (iOS & Web Browsers): Users on iOS devices or those who prefer running simulations without local CLI setup can navigate to colab.research.google.com, create a new notebook, paste the Python script into an initial execution cell, and run it directly in the cloud.

4 Complete Python Implementation (double_pendulum.py)

The script below computes the simulation for 20.0 seconds (500 frames at 25 FPS). It simulates two double pendulums initialized with an angular difference of **exactly** 1.0° , rendering their trajectories in high-contrast neon colors on a 9:16 vertical canvas with persistent fading ribbons:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation, PillowWriter
4
5 # =====
6 # 1. PHYSICAL CONSTANTS & EQUATIONS OF MOTION
7 # =====
8 G = 9.81 # Gravitational acceleration (m/s^2)
9 L1, L2 = 1.0, 1.0 # Pendulum rod lengths (meters)
10 M1, M2 = 1.0, 1.0 # Bob masses (kg)
11
12 def derivatives(state):
13     """
14     Computes angular accelerations using coupled Lagrangian mechanics.
15     state = [theta1, omega1, theta2, omega2]
16     """
17     t1, w1, t2, w2 = state
18     delta = t1 - t2
19     den = 2 * M1 + M2 - M2 * np.cos(2 * t1 - 2 * t2)
20
21     # Angular acceleration for bob 1 (alpha1)
22     num1 = (-G * (2 * M1 + M2) * np.sin(t1)
23             - M2 * G * np.sin(t1 - 2 * t2)
24             - 2 * np.sin(delta) * M2 * (w2**2 * L2 + w1**2 * L1 * np.cos(delta)))
25     alpha1 = num1 / (L1 * den)
26
27     # Angular acceleration for bob 2 (alpha2)
28     num2 = (2 * np.sin(delta) * (w1**2 * L1 * (M1 + M2)
29             + G * (M1 + M2) * np.cos(t1)
30             + w2**2 * L2 * M2 * np.cos(delta)))
```

```

81     alpha2 = num2 / (L2 * den)
82
83     return np.array([w1, alpha1, w2, alpha2])
84
85 def rk4_step(state, dt):
86     """4th-order Runge-Kutta numerical integration step."""
87     k1 = derivatives(state)
88     k2 = derivatives(state + 0.5 * dt * k1)
89     k3 = derivatives(state + 0.5 * dt * k2)
90     k4 = derivatives(state + dt * k3)
91     return state + (dt / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)
92
93 # =====
94 # 2. SIMULATION TIMING & INITIAL PERTURBATION
95 # =====
96 FPS = 25
97 DURATION_SEC = 20 # Exact 20 seconds duration
98 TOTAL_FRAMES = DURATION_SEC * FPS # Exactly 500 frames
99 DT_FRAME = 1.0 / FPS
100 SUB_STEPS = 5 # 5 micro-steps per frame for stability
101 DT_SUB = DT_FRAME / SUB_STEPS
102
103 # Initial angles: Released from horizontal position (90 degrees)
104 INITIAL_THETA1 = np.pi / 2
105 INITIAL_THETA2 = np.pi / 2
106
107 # Exact 1.0 degree perturbation for system B
108 DELTA_THETA = np.radians(1.0)
109
110 state_a = np.array([INITIAL_THETA1, 0.0, INITIAL_THETA2, 0.0])
111 state_b = np.array([INITIAL_THETA1 + DELTA_THETA, 0.0, INITIAL_THETA2, 0.0])
112
113 history_a = np.zeros((TOTAL_FRAMES, 4))
114 history_b = np.zeros((TOTAL_FRAMES, 4))
115
116 print(f"Integrating physics for {DURATION_SEC}s ({TOTAL_FRAMES} frames)...")
117 for frame in range(TOTAL_FRAMES):
118     history_a[frame] = state_a
119     history_b[frame] = state_b
120     for _ in range(SUB_STEPS):
121         state_a = rk4_step(state_a, DT_SUB)
122         state_b = rk4_step(state_b, DT_SUB)
123
124 # Convert polar angles to Cartesian coordinates (X, Y)
125 # System A (Cyan)
126 x1_a = L1 * np.sin(history_a[:, 0])
127 y1_a = -L1 * np.cos(history_a[:, 0])
128 x2_a = x1_a + L2 * np.sin(history_a[:, 2])
129 y2_a = y1_a - L2 * np.cos(history_a[:, 2])
130
131 # System B (Magenta)
132 x1_b = L1 * np.sin(history_b[:, 0])
133 y1_b = -L1 * np.cos(history_b[:, 0])
134 x2_b = x1_b + L2 * np.sin(history_b[:, 2])
135 y2_b = y1_b - L2 * np.cos(history_b[:, 2])
136
137 # =====

```

```

88 # 3. VISUALIZATION (VERTICAL 9:16 FORMAT)
89 # =====
90 fig, ax = plt.subplots(figsize=(6, 10.6), facecolor="#020202")
91 ax.set_facecolor("#020202")
92 ax.set_xlim(-2.3, 2.3)
93 ax.set_ylim(-2.5, 1.5)
94 ax.axis("off")
95
96 # Fixed center pivot anchor
97 ax.plot(0, 0, marker="o", markersize=6, color="#FFFFFF", alpha=0.9)
98
99 # System A (Cyan - Reference Pendulum)
100 trail_a, = ax.plot([], [], color="#00E5FF", lw=1.6, alpha=0.75, label="Base Pendulum")
101 line_a, = ax.plot([], [], "o-", color="#80F0FF", lw=2.2, markersize=7, zorder=4)
102
103 # System B (Neon Magenta - Perturbed by 1.0 Degree)
104 trail_b, = ax.plot([], [], color="#FF0077", lw=1.6, alpha=0.75, label="Perturbed (+1.0
105 )")
106 line_b, = ax.plot([], [], "o-", color="#FFA0CF", lw=2.2, markersize=7, zorder=5)
107
108 # Digital HUD overlay metrics
109 time_text = ax.text(0.05, 0.94, "", transform=ax.transAxes, color="#FFFFFF",
110                    fontsize=12, family="monospace", weight="bold")
111 delta_text = ax.text(0.05, 0.90, "Delta Theta: 1.0",
112                    transform=ax.transAxes, color="#FFD700", fontsize=10, family="
113                    monospace")
114
115 # High-contrast scientific legend
116 ax.legend(loc="upper right", facecolor="#101010", edgecolor="#2B2B2B",
117          labelcolor="#E0E0E0", fontsize=9, framealpha=0.85)
118
119 TRAIL_LENGTH = 100 # 4 seconds of active trajectory persistence
120
121 def update(frame):
122     start_idx = max(0, frame - TRAIL_LENGTH)
123
124     # Update dynamic trajectories
125     trail_a.set_data(x2_a[start_idx:frame], y2_a[start_idx:frame])
126     trail_b.set_data(x2_b[start_idx:frame], y2_b[start_idx:frame])
127
128     # Update rigid physical links
129     line_a.set_data([0, x1_a[frame], x2_a[frame]], [0, y1_a[frame], y2_a[frame]])
130     line_b.set_data([0, x1_b[frame], x2_b[frame]], [0, y1_b[frame], y2_b[frame]])
131
132     # Update real-time HUD clock
133     current_time = frame * DT_FRAME
134     time_text.set_text(f"Time: {current_time:.1f}s / {DURATION_SEC}.0s")
135
136     return trail_a, trail_b, line_a, line_b, time_text
137
138 ani = FuncAnimation(fig, update, frames=TOTAL_FRAMES, interval=1000/FPS, blit=True)
139
140 # =====
141 # 4. EXPORT ANIMATION
142 # =====
143 output_filename = "double_pendulum_chaos_20s.gif"
144 print(f"Rendering {TOTAL_FRAMES} frames ({DURATION_SEC}s)...")

```

```

143 ani.save(output_filename, writer=PillowWriter(fps=FPS))
144 print(f"Done! File saved successfully as: {output_filename}")

```

5 Step-by-Step Mobile Execution & Storage Export

1. **Create the Script File:** Inside the Ubuntu CLI, initialize a new Python file:

```
nano double_pendulum.py
```

2. **Paste & Save:** Paste the source code into the editor. Press **Ctrl + O** followed by **Enter** to write the buffer, then **Ctrl + X** to exit.
3. **Execute Numerical Engine:** Run the simulation script:

```
python3 double_pendulum.py
```

The CPU will compute the 500 physics states in fractions of a second, followed by visual frame rasterization. The process completes with the terminal confirmation message **Done!**.

4. **Transfer to Local Device Storage:** To view the rendered **.gif** animation inside the mobile gallery or file manager, export it to the shared user storage:

```

# Exit Ubuntu root shell back to primary Termux environment
exit

# Copy the generated asset to the system Download directory
cp ~/ubuntu/root/double_pendulum_chaos_20s.gif /sdcard/Download/

```

Navigate to the **Download** directory within your mobile file explorer to inspect the animation.

6 Physical Analysis of the Simulation Output

When analyzing the rendered output, observe the three distinct dynamical regimes:

[0.0s - 3.5s]	→	[3.5s - 5.0s]	→	[5.0s - 20.0s]
Phase Coherence	→	Bifurcation Threshold	→	Deterministic Chaos
(Overlapping Paths)		(Separation Begins)		(Completely Divergent Orbits)

1. **Phase Coherence (0.0s – 3.5s):** Despite the intentional 1.0° angular offset, both pendulums move in visual synchrony. The cyan and magenta trails overlap almost completely, rendering a unified composite hue.
2. **Bifurcation Threshold ($\approx 3.8s$):** Near the 3.8s mark, the primary bob in one system acquires just enough kinetic momentum to invert over the apex equilibrium, while the secondary system falls fractions of a joule short.

3. **Unbounded Chaos (5.0s – 20.0s):** One pendulum executes a full 360° loop while the other reverses its oscillation at the bottom. Past the 6.0s boundary, an observer inspecting the state vectors cannot infer that both trajectories originated from nearly identical initial conditions.

7 Computational Experiments & Research Challenges

To explore deeper dynamics of the phase space, modify the simulation parameters inside `double_pendulum.py`:

Challenge 1: The Lyapunov Threshold Test

Reduce the initial perturbation parameter from 1.0° to 0.0001 rad:

```
1 DELTA_THETA = 0.0001
```

Rerun the integration. Document the exact second at which macroscopic spatial separation occurs.

Challenge 2: Mass Asymmetry Breaking

Increase the secondary bob's mass tenfold ($M2 = 10.0$). Observe whether the amplified whip effect accelerates or delays the onset of bifurcation.

Challenge 3: Geometric Long-Exposure Attractor

Extend the trail persistence parameter from 100 to 500 frames:

```
1 TRAIL_LENGTH = 500
```

This retains the entire 20-second trajectory history, turning the chaotic path into a dense topological phase portrait.

Developed by **Poria Azadi** • Follow the educational series on Telegram: [@the_maze2022](https://t.me/the_maze2022)